

Aceleración GPU con PyTorch

¿Cuándo Vale la Pena?



La Pregunta que Vamos a Responder

Una **multiplicación de matrices** que tarda **1 segundo en CPU** tarda **5 milisegundos en GPU** — ¿o no?

- El mismo código. La misma operación. Hardware diferente.
- ¿Qué cambia? ¿Qué queda oculto? ¿Cuándo falla?

```
# Evaluado para distintos tamaños: N = 10, 100, 1000, 5000, 10000
A = torch.randn(N, N)
B = torch.randn(N, N)
C = torch.mm(A, B)    # CPU - nuestro punto de partida
```

5 Preguntas

1. ¿Qué problema resuelve la GPU?
2. ¿Qué tan fácil es migrar a GPU?
3. ¿Qué cambia realmente?
4. ¿Qué costos aparecen?
5. ¿Cuándo NO ves ningún beneficio?

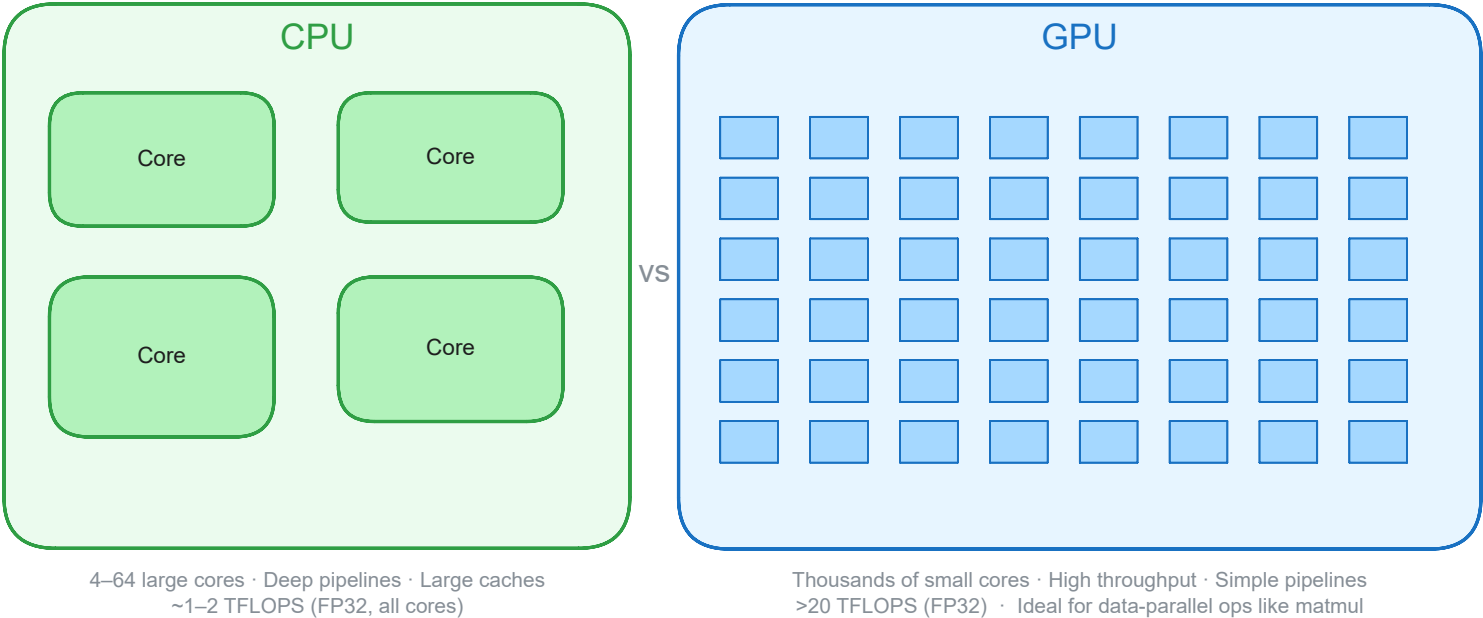
Conclusión: cuándo vale la pena

CPU vs GPU — Paralelismo

CPU	GPU
4–64 núcleos, optimizado para baja latencia	Miles de núcleos de cómputo, optimizado para throughput
Cada núcleo: rápido, pipeline profundo, caché grande	Cada núcleo: más simple, pero masivamente paralelo
Bueno para código ramificado y secuencial	Bueno para código data-paralelo, misma operación

The Same Operation — Different Hardware

`C = torch.mm(A, B)` ← same call, same operation, dispatched to different hardware



P1: ¿Qué Problema Resuelve la GPU?

El cuello de botella en `torch.mm`: $C[i, j] = \sum_k A[i, k] \cdot B[k, j]$ – para $N=5000$, **125 mil millones de FMA** (~250 GFLOPs en convención FP32), ejecutadas con BLAS multihilo optimizado en CPU.

Base CPU: Midiendo la escalabilidad con N

```
import torch, time

def time_mm_cpu(N, runs=10):
    A, B = torch.randn(N, N), torch.randn(N, N)
    for _ in range(3): torch.mm(A, B) # calentar caché CPU
    t0 = time.perf_counter()
    for _ in range(runs): torch.mm(A, B)
    return (time.perf_counter() - t0) / runs * 1000 # ms

for N in [10, 100, 500, 1000, 5000]:
    print(f"N={N:5d}: {time_mm_cpu(N):.3f} ms")
```

Cada elemento de salida es independiente:

```
# Conceptualmente:  
for i in range(N):  
    for j in range(N):  
        C[i,j] = sum(A[i,:] * B[:,j])    # ¡independiente!
```

La GPU procesa una gran cantidad de los N^2 productos internos en paralelo mediante kernels por bloques (tiling).

- GPU de alta gama (Tensor Cores / HW Dedicado): **>300 TFLOPS (FP16/BF16) · ~20 TFLOPS (FP32)**
 - CPU de alta gama (Multi-Threaded MKL/OpenBLAS): **~1–2 TFLOPS (FP32, todos los núcleos)**
- **~10–20x ventaja teórica en FP32, escalando a >150x mediante unidades matriciales dedicadas**

P2: ¿Qué Tan Fácil Es Migrar a GPU?

La respuesta: una línea — `.to(device)`.

```
import torch

device = torch.accelerator.current_accelerator()
# detecta el acelerador disponible
N = 1000
A = torch.randn(N, N)
B = torch.randn(N, N)

# CPU - código original
C_cpu = torch.mm(A, B)

# Acelerador - mover tensores al dispositivo
C_acc = torch.mm(A.to(device), B.to(device))
```

Patrón agnóstico de dispositivo (recomendado)

```
import torch

# Funciona en cualquier hardware
device = torch.accelerator.current_accelerator()
print(f"Usando: {device}")

N = 1000
# Crear tensores directamente en el dispositivo destino
# sin costo de transferencia
A = torch.randn(N, N, device=device)
B = torch.randn(N, N, device=device)

# misma llamada - PyTorch despacha el kernel correcto
C = torch.mm(A, B)
print(C.device)
```

- Sin reescritura. Sin API específica del proveedor. Un objeto de dispositivo.

Tres errores que cometen los principiantes al hacer benchmarks de GPU:

1. **Sin calentamiento** — la primera ejecución incluye la inicialización del contexto del acelerador (~500 ms)
2. **Sin synchronize** — las operaciones del acelerador son async; mides el *lanzamiento*, no la *ejecución*
3. **Ejecución única** — alta varianza; siempre promedia sobre múltiples ejecuciones

Función de benchmark correcta

```
import torch, time

def benchmark(N, device, warmup=3, runs=10):
    A = torch.randn(N, N, device=device)
    B = torch.randn(N, N, device=device)
    # calentar cachés y JIT
    for _ in range(warmup):
        torch.mm(A, B)
    # asegurar que el calentamiento esté completo
    torch.accelerator.synchronize()
    t0 = time.perf_counter()
    for _ in range(runs):
        torch.mm(A, B)
    # asegurar que todo el trabajo esté terminado
    torch.accelerator.synchronize()
    # ms por llamada
    return (time.perf_counter() - t0) / runs * 1000
```

Resultados del Benchmark

N	CPU (ms)	GPU (ms)	Speedup
10	0.01	0.05	0.2x (¡GPU más lento!)
100	0.15	0.06	2.5x ← Umbral / Punto de Cruce
500	3.2	0.09	35.6x
1000	12	0.4	30x
5000	980	8	122x
10000	~8000	45	~180x

Valores representativos – CPU moderna de 8 núcleos / GPU de cómputo · FP32

El speedup no es constante – depende de N.

Por Qué el Speedup Crece con N

Cómputo requerido: $O(N^3)$

Ancho de banda de memoria: $O(N^2)$

Para N pequeño: **el overhead domina** (lanzamiento de kernel, transferencia de memoria)

Para N grande: **el cómputo domina** → la utilización de la GPU llega al 100%

N	10	50	100	500	1000	5000	10000
Speedup	0.2x	1.1x	2.5x	35x	30x	122x	180x
Régimen	overhead	overhead	cruce	cómputo	cómputo	cómputo	cómputo

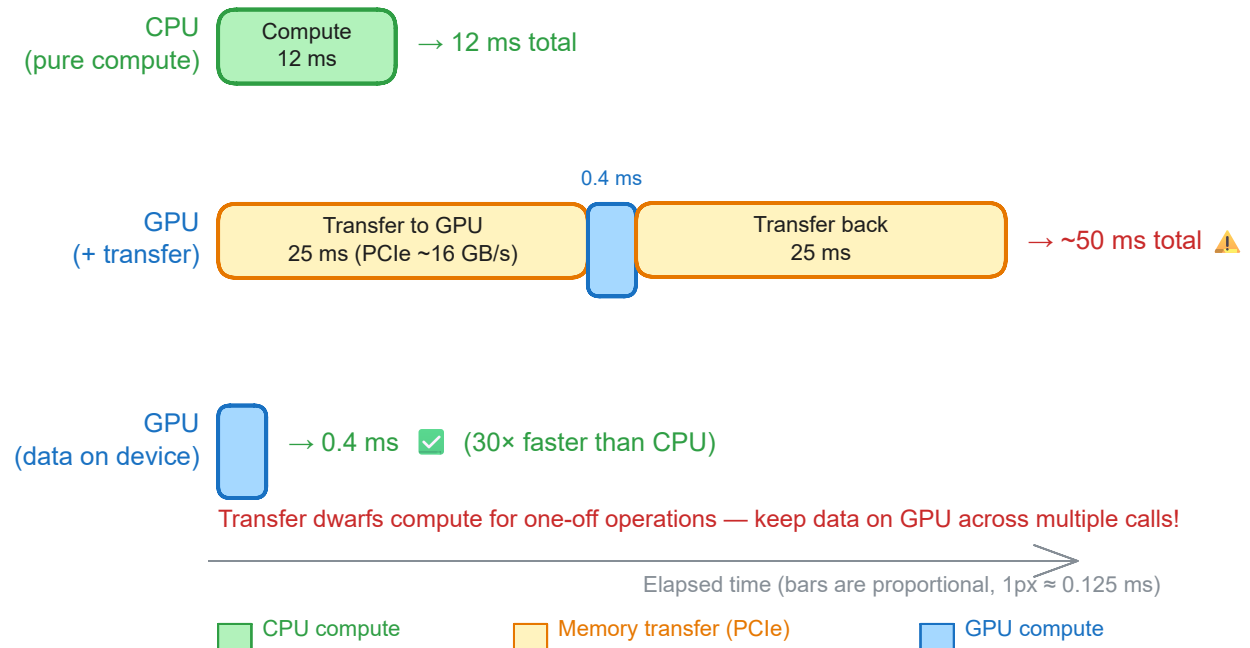
La GPU es un auto de carreras. No la uses para estacionar.

P4: ¿Qué Costos Aparecen?

Tres costos ocultos que debes considerar:

1. **Transferencia de datos** — mover tensores de memoria principal a memoria del acelerador
2. **Overhead de lanzamiento de kernel** — latencia del despacho CPU → acelerador
3. **Calentamiento en la primera ejecución** — inicialización del contexto del acelerador + compilación de kernels

End-to-End Latency — `torch.mm(A,B)`, $N=1000$



Costo 1: Transferencia de Datos

```
import torch, time

device = torch.accelerator.current_accelerator()
N = 5000
A = torch.randn(N, N)      # en CPU
torch.accelerator.synchronize() # Medir solo la transferencia
t0 = time.perf_counter()
A_dev = A.to(device)       # memoria del host → acelerador
torch.accelerator.synchronize()
transfer_ms = (time.perf_counter() - t0) * 1000

print(f"Tamaño de datos: {A.nbytes / 1e6:.1f} MB")
print(f"Transferencia: {transfer_ms:.1f} ms")
print(f"Cómputo:      ~8 ms")      # del benchmark
```

Tamaño de datos: 100.0 MB
Transferencia: ~25 ms ← ¡3x más lento que el cómputo!

La transferencia puede eclipsar el cómputo en operaciones de uso único.

Costo 2: Overhead de Lanzamiento de Kernel

```
import torch, time

device = torch.accelerator.current_accelerator()
A = torch.randn(10, 10, device=device)  # matriz diminuta
B = torch.randn(10, 10, device=device)

# Medir una sola operación pequeña
torch.accelerator.synchronize()
t0 = time.perf_counter()
C = torch.mm(A, B)
torch.accelerator.synchronize()
print(f"N=10 en acelerador: {(time.perf_counter()-t0)*1000:.3f} ms")
```

N=10 en acelerador: 0.050 ms ← 5x más lento que CPU para la misma op

Cada kernel del acelerador tiene un costo fijo de lanzamiento de ~5–50 μ s.

Costo 3: Calentamiento en la Primera Ejecución

```
import torch, time

device = torch.accelerator.current_accelerator()
N = 1000
A = torch.randn(N, N, device=device)
B = torch.randn(N, N, device=device)

for i in range(5):
    torch.accelerator.synchronize()
    t0 = time.perf_counter()
    C = torch.mm(A, B)
    torch.accelerator.synchronize()
    print(f"Ejecución {i}: {(time.perf_counter()-t0)*1000:.1f} ms")
```

```
Ejecución 0: 850 ms ← inicialización del contexto + selección del plan de kernel
Ejecución 1: 0.4 ms
Ejecución 2: 0.4 ms
Ejecución 3: 0.4 ms
Ejecución 4: 0.4 ms
```

Siempre calienta antes de hacer benchmark. Siempre.

Tres situaciones comunes donde la GPU te perjudica:

Situación	Por qué pierde la GPU
Tensores pequeños ($N < \sim 200$)	Lanzamiento de kernel > cómputo
Operación de uso único	Costo de transferencia > ahorro en cómputo
Operaciones secuenciales	La GPU está inactiva entre cada paso

Situación 1: Tensores pequeños

```
import torch, time

cpu = torch.device("cpu")
acc = torch.accelerator.current_accelerator()

for N in [10, 50, 100, 200, 500]:
    t_cpu = benchmark(N, cpu)
    t_acc = benchmark(N, acc) # incluye sync, no transferencia
    winner = "Acelerador" if t_acc < t_cpu else "CPU"
    print(f"N={N:4d}: CPU={t_cpu:.3f}ms ACC={t_acc:.3f}ms → {winner}")
```

N= 10:	CPU=0.010ms	ACC=0.050ms	→ CPU
N= 50:	CPU=0.060ms	ACC=0.055ms	→ CPU
N= 100:	CPU=0.15ms	ACC=0.060ms	→ Acelerador
N= 200:	CPU=0.80ms	ACC=0.065ms	→ Acelerador
N= 500:	CPU=3.2ms	ACC=0.090ms	→ Acelerador

Punto de cruce: ~N=100 en este hardware.

Situación 2: El overhead de transferencia domina

```
import torch, time

device = torch.accelerator.current_accelerator()

def end_to_end_acc(N):
    A = torch.randn(N, N)    # en CPU
    B = torch.randn(N, N)
    t0 = time.perf_counter()
    # transferencia + cómputo + transferencia de vuelta
    C = torch.mm(A.to(device), B.to(device)).cpu()
    torch.accelerator.synchronize()
    return (time.perf_counter() - t0) * 1000

# Comparar con CPU puro
```

Para N=1000: CPU=12ms · Cómputo acelerador=0.4ms · **Acelerador extremo a extremo=28ms**

Si solo ejecutas la operación una vez y los datos están en CPU: no muevas al acelerador.

Situación 3: Cadena de dependencia secuencial

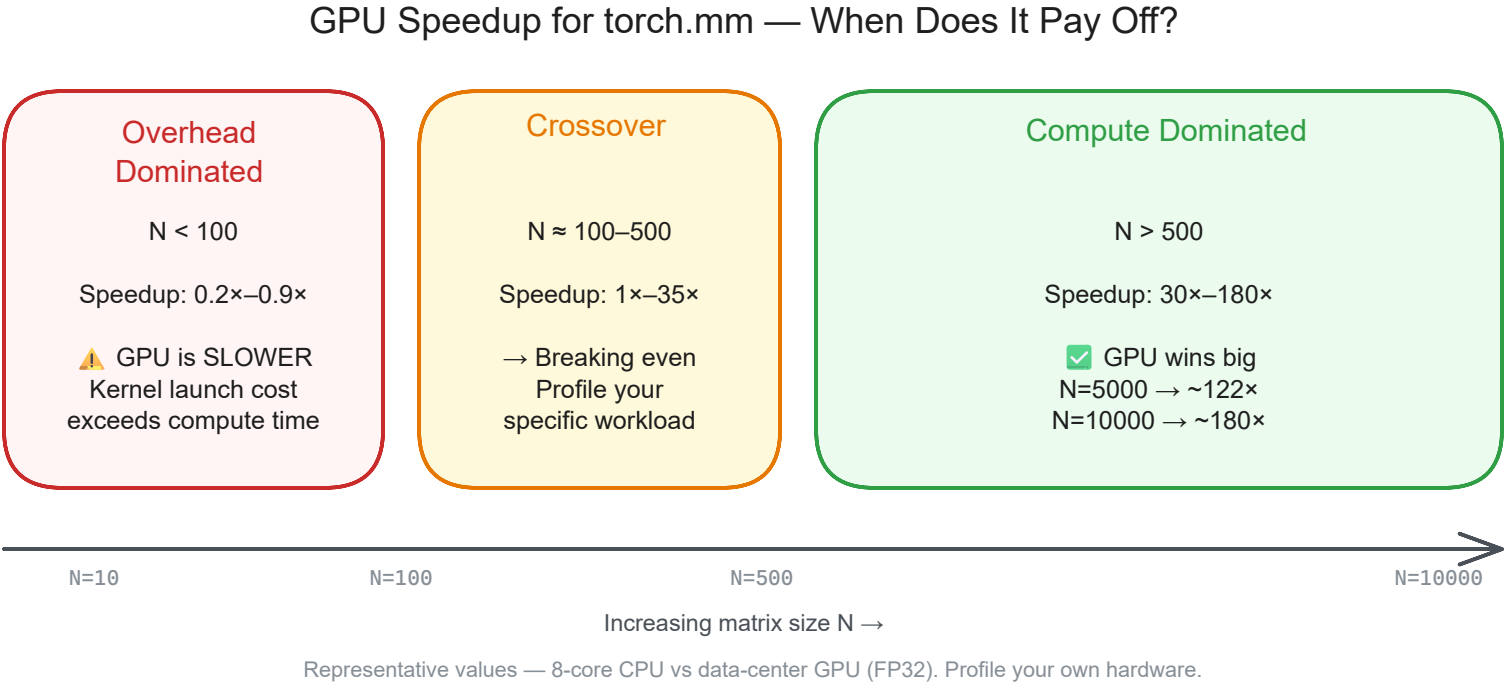
```
# MAL: el acelerador está inactivo
# sync forzado en cada iteración
for step in range(100):
    # en el acelerador
    x = torch.mm(x, W)
    # .item() fuerza sync!
    if x.max().item() > threshold:
        break
```

```
# MEJOR: mantener todo el cómputo en el acelerador,
# sync solo al final
for step in range(100):
    # todo en el acelerador, sin sync
    x = torch.mm(x, W)
# transferencia única al final
result = x.cpu()
```

`.item()`, `print(tensor)` y `if` de Python sobre tensores del acelerador fuerzan sincronización.

Conclusión: Tabla de Decisión

Tu escenario	Recomendación
N > 500, ejecutado muchas veces	Usa GPU — gran speedup
N < 100, cualquier frecuencia	Quédate en CPU
Datos ya en GPU (pipeline)	Usa GPU — sin costo de transferencia
Operación única, datos en CPU	CPU puede ser más rápido de extremo a extremo
Operaciones secuenciales con control Python	Perfila primero
Bucle de entrenamiento (muchos batches)	GPU — amortiza la transferencia



La GPU es beneficiosa cuando:

$$\frac{\text{cómputo ahorrado en GPU}}{\text{overhead (transferencia + lanzamiento)}} > 1$$

- **Agranda el numerador:** matrices grandes, muchas operaciones
- **Reduce el denominador:** mantén datos en GPU, agrupa operaciones

"Una multiplicación de matrices que tarda **1 segundo en CPU** tarda **5 milisegundos en GPU** — ¿o no?"

Sí — pero solo si:

- La matriz es grande ($N > \sim 500$)
- Los datos ya están en GPU (o ejecutas muchas veces)
- Calientas la GPU primero

Para $N=10000$ en un bucle de entrenamiento: $\sim 180x$ de speedup.

Para $N=10$ llamado una vez con datos en CPU: la GPU es $5x$ más lenta.

Ejecuta esto en Colab (runtime GPU):

```
import torch, time

def benchmark(N, device, warmup=3, runs=20):
    A = torch.randn(N, N, device=device)
    B = torch.randn(N, N, device=device)
    for _ in range(warmup): torch.mm(A, B)
    torch.accelerator.synchronize()
    t0 = time.perf_counter()
    for _ in range(runs): torch.mm(A, B)
    torch.accelerator.synchronize()
    return (time.perf_counter() - t0) / runs * 1000

cpu = torch.device("cpu")
acc = torch.accelerator.current_accelerator()
print(f"PyTorch: {torch.__version__} | Acelerador: {acc}")
for N in [10, 100, 500, 1000, 5000]:
    tc = benchmark(N, cpu)
    ta = benchmark(N, acc)
    print(f"N={N:5d} CPU={tc:8.2f}ms ACC={ta:7.3f}ms speedup={tc/ta:6.1f}x")
```


1. **El acelerador gana en trabajo grande, paralelo y repetido** — no en ops pequeñas o de uso único
2. `.to(device)` es una línea — pero considera los costos de transferencia y calentamiento
3. **Siempre sincroniza antes de medir** — las operaciones del acelerador son asíncronas
4. **El punto de cruce** es $\sim N=100-200$ para `torch.mm` en hardware moderno
5. **Perfila tu carga de trabajo real** — la intuición puede engañarte

- API `torch.accelerator` de PyTorch: docs.pytorch.org/docs/stable/accelerator.html
- Guía de ajuste de rendimiento de PyTorch:
docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

La migración en una línea:

```
device = torch.accelerator.current_accelerator()
```

```
device = torch.accelerator.current_accelerator()  
A = A.to(device)  
B = B.to(device)  
C = torch.mm(A, B)
```

¿Preguntas?